Звіт з тестування безпеки вебзастосунку

Session Management Testing та Input Validation Testing

Мета та умови роботи

Метою роботи було перевірити керування сесіями й обробку введення у навчальному вебзастосунку, порівняти Vulnerable та Safe і підтвердити висновки практичними перевірками. Я тестувала обліковий запис user7 на https://web-production-lab.up.railway.app/

Використовувала Chrome, DevTools, Burp Suite і curl, а для незалежних сесій відкривала звичайне вікно браузера та інкогніто.

Для Command Injection, Path Traversal та SSTI передбачалась симуляцію, тому результати цих перевірок не прирівнюю до виконання команд або читання реальних файлів сервера.

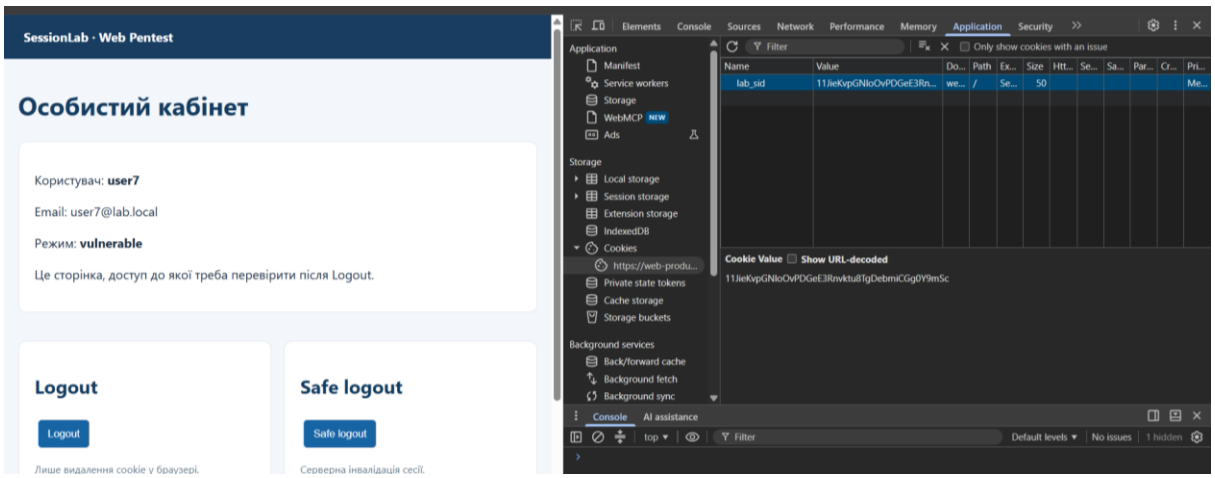
Session Management Testing

1 Session ID та Session Fixation

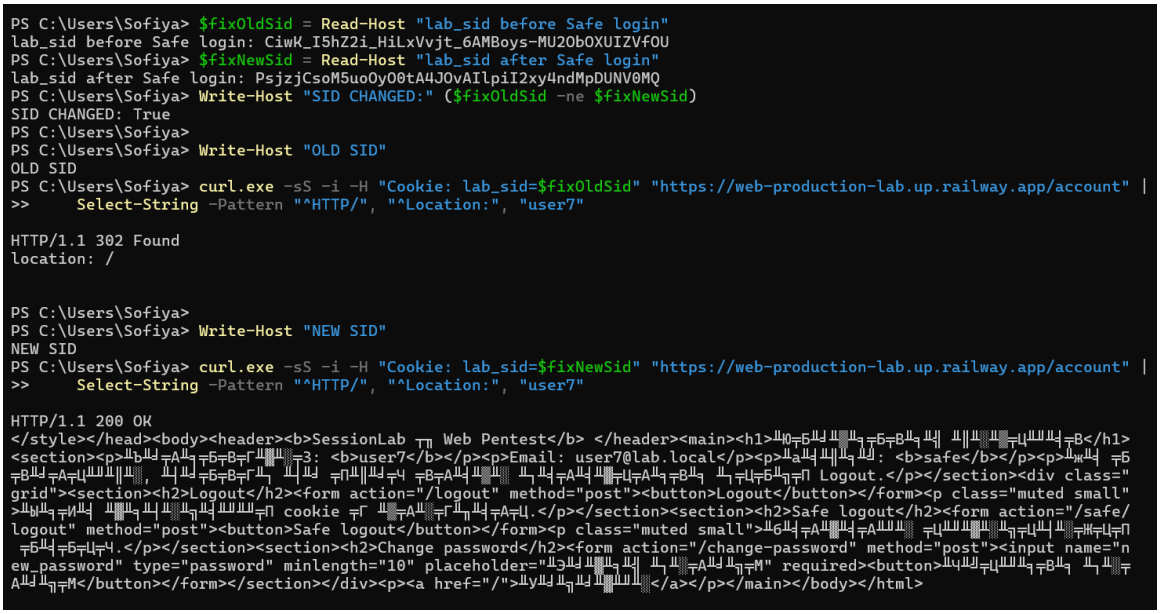
Що перевіряла. Чи змінюється lab_sid після авторизації та чи дає попередній sid доступ до кабінету.

Хід перевірки. До входу скопіювала lab_sid, увійшла як user7 та знову переглянула cookie, щоб порівняти значення до й після авторизації. У Vulnerable підставила збережений sid в інкогніто й відкрила /account, щоб перевірити доступ із незалежного вікна. Для Safe зберегла sid до входу та новий sid після входу, а потім окремо надіслала GET /account з кожним значенням cookie, не виконуючи Logout або зміну пароля між запитами.

Перевірка	**Vulnerable**	**Safe**
SID до і після входу	Однаковий	Змінився
Доступ зі старим SID після входу	В інкогніто відкрився кабінет user7	302 Found; Location: /
Доступ з поточним SID	Кабінет доступний	200 OK; кабінет user7; режим safe



Vulnerable: кабінет user7 відкрився в інкогніто після підстановки sid, який не змінився при вході.



Safe: SID CHANGED: True, старий sid отримав 302 з Location: /, а новий повернув 200 і кабінет user7.

Результат: у Vulnerable sid до входу і після входу однаковий, і цей sid відкрив кабінет user7 в інкогніто. У Safe sid після входу змінився (SID CHANGED: True), старий sid отримав 302 на головну сторінку, а новий отримав 200 і кабінет user7.

Причина: у Vulnerable сервер після успішного входу не створює нову сесію, а залишає ту, яка була видана ще до авторизації, тому ідентифікатор, відомий до входу, після входу автоматично стає авторизованим. У Safe сервер після входу видає новий SID, а старий робить недійсним.

Ризик (High): у реальному застосунку нападник може отримати або нав'язати жертві свій sid, і коли жертва увійде, цей самий sid дасть нападнику доступ до її акаунта без знання пароля. Це може привести до повного захоплення облікового запису.

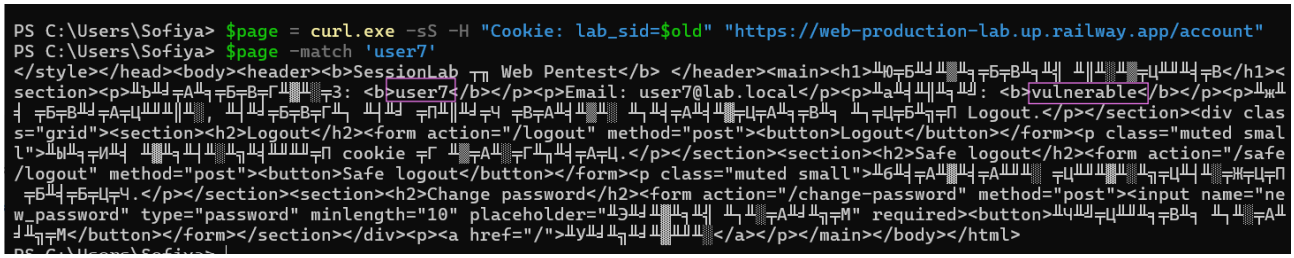
Рекомендація: після успішного входу створювати нову сесію з новим sid і знищувати стару. Після виправлення повторити тест: sid до і після входу має відрізнятись, а старий sid має отримувати 302.

2 Завершення сесій після Logout

Що перевіряла. Чи припиняє сервер приймати sid після виходу -для цього потрібно повторно надіслати стару cookie, так як саме повернення браузера на головну ще не доводить завершення сесії.

У Vulnerable зберегла поточний lab_sid і перевірила GET /account до виходу, коли кабінет user7 був доступний. Натиснула Logout у тому самому вікні та повторила запит до /account зі збереженим sid через curl. Створила нову Safe-сесію, перевірила доступ до виходу, натиснула Safe logout і повторила запит із тим самим збереженим sid. Для повторного запиту значення cookie не змінювала.

Запит GET /account зі збереженим sid	**Vulnerable**	**Safe**
До виходу	200 OK; кабінет user7	200 OK; user7; safe
Після відповідного Logout	Кабінет user7 залишився доступним	302 Found; Location: /



Vulnerable: запит зі старим sid після звичайного Logout повернув HTML кабінету з user7 та vulnerable.



Safe Logout: запит із тим самим sid до виходу отримав 200, а після виходу отримав 302 та Location: /.

Результат: у Vulnerable старий sid після звичайного Logout далі відкриває кабінет user7 (200 OK), а в Safe після Safe logout той самий SID отримує 302 на головну сторінку.

Причина: кнопка Logout у Vulnerable лише видаляє cookie в браузері (це написано на самій сторінці), а сесія на сервері лишається активною, тому з відомим sid можна далі користуватись акаунтом. Safe logout знищує сесію на сервері, тому старий sid перестає працювати.

Ризик (High): якщо sid раніше потрапив до нападника (наприклад, через XSS, перехоплення трафіку або спільний комп'ютер), то вихід власника з акаунта не закриє нападнику доступ, і він зможе користуватись сесією довго після виходу.

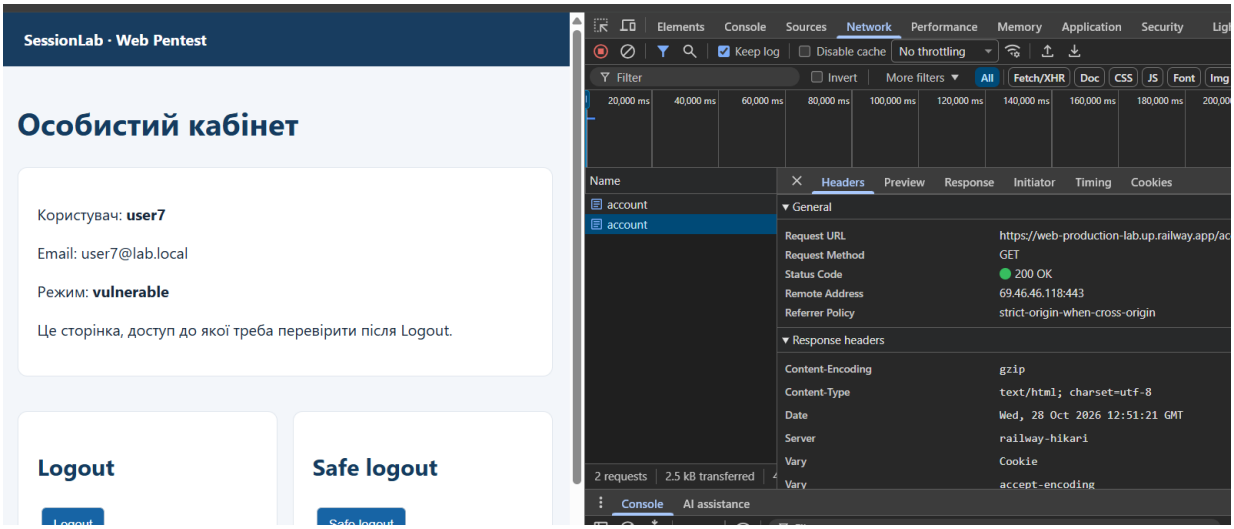
Обмеження: я порівнювала саме звичайний Logout і Safe logout, тому не переношу висновок на інші комбінації кнопки та режиму.

Рекомендація: при виході видаляти сесію на сервері і очищати cookie в браузері. Після виправлення повторити запит зі старим SID і перевірити, що він отримує 302 і не повертає захищений вміст.

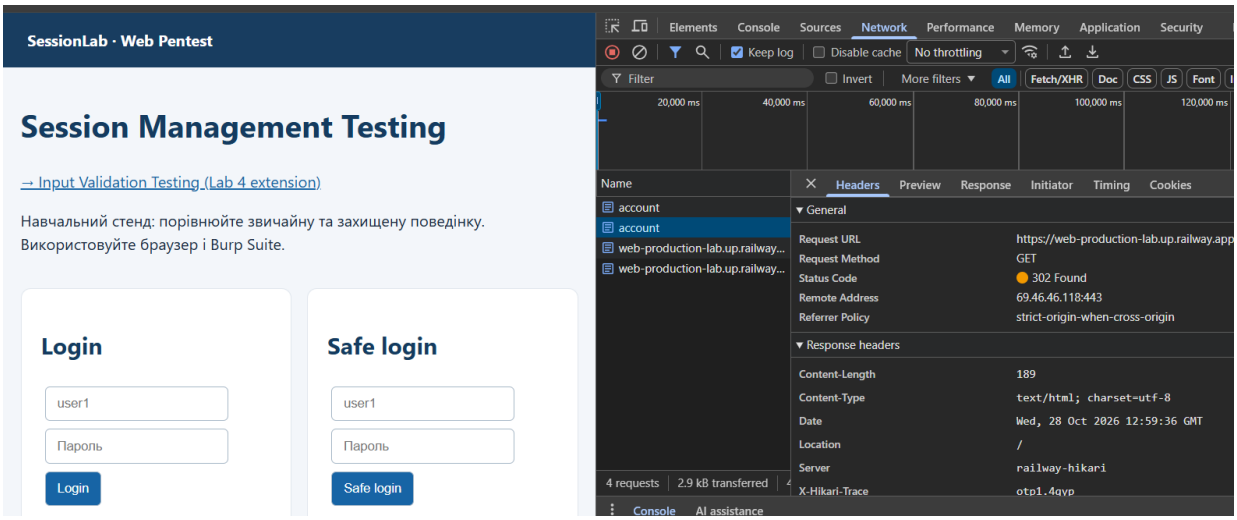
3 Активні сесії після зміни пароля

Що перевіряла. Чи залишається інша активна сесія після зміни пароля облікового запису.

Увійшла як user7 у двох незалежних вікнах A і B та перевірила, що обидва відкривають /account. У вікні A змінила пароль через форму. POST /change-password повернув 200 OK та повідомлення Password changed. Без повторного входу оновила /account у вікні B й записала статус та вміст відповіді. Повторила цю послідовність для Safe, використовуючи дві Safe-сесії.



Vulnerable: після зміни пароля у вікні A інша сесія B продовжила отримувати кабінет user7 з 200 OK.



Safe: після зміни пароля запит /account іншої сесії отримав 302 Found та Location: /.

Результат: у Vulnerable після зміни пароля у вікні A вікно B продовжило відкривати кабінет user7 (200 OK), а в Safe вікно B після зміни пароля отримало 302 на головну сторінку.

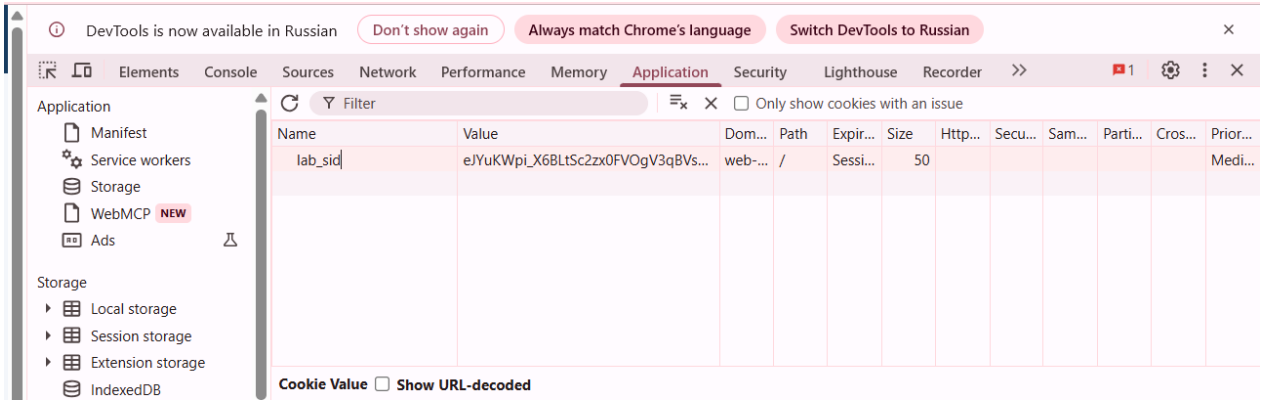
Причина: у Vulnerable сервер при зміні пароля не завершує інші активні сесії цього користувача, а в Safe завершує.

Ризик (High): якщо нападник уже має активну сесію (наприклад, викрав sid), то власник акаунта змінює пароль і вважає, що відновив контроль, а нападник усе одно лишається в акаунті. Зміна пароля не виконує свою головну задачу.

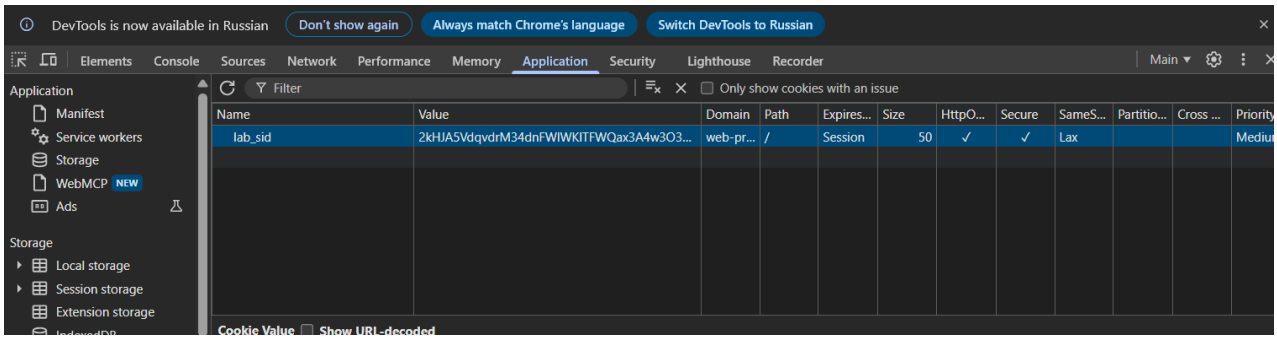
Рекомендація: після зміни пароля завершувати всі інші сесії користувача і вимагати повторний вхід, а поточну сесію оновлювати новим sid. Після виправлення повторити тест із двома вікнами і перевірити, що друге вікно отримує 302.

4 Атрибути безпеки Cookie

Що перевіряла. Як налаштована cookie lab_sid, за допомогою якої браузер передає ідентифікатор сесії.



Vulnerable: у таблиці lab_sid немає позначок HttpOnly і Secure та явного значення SameSite.



Safe: для lab_sid встановлені HttpOnly, Secure і SameSite=Lax.

Результат: у Vulnerable cookie lab_sid не має HttpOnly і Secure, а SameSite не заданий. У Safe для lab_sid встановлені HttpOnly, Secure і SameSite=Lax.

Причина: у Vulnerable сервер при створенні cookie не додає захисні атрибути, а в Safe додає. HttpOnly не дає JavaScript читати cookie, Secure не дозволяє надсилати її незахищеним HTTP, а SameSite обмежує відправку cookie у міжсайтових запитах.

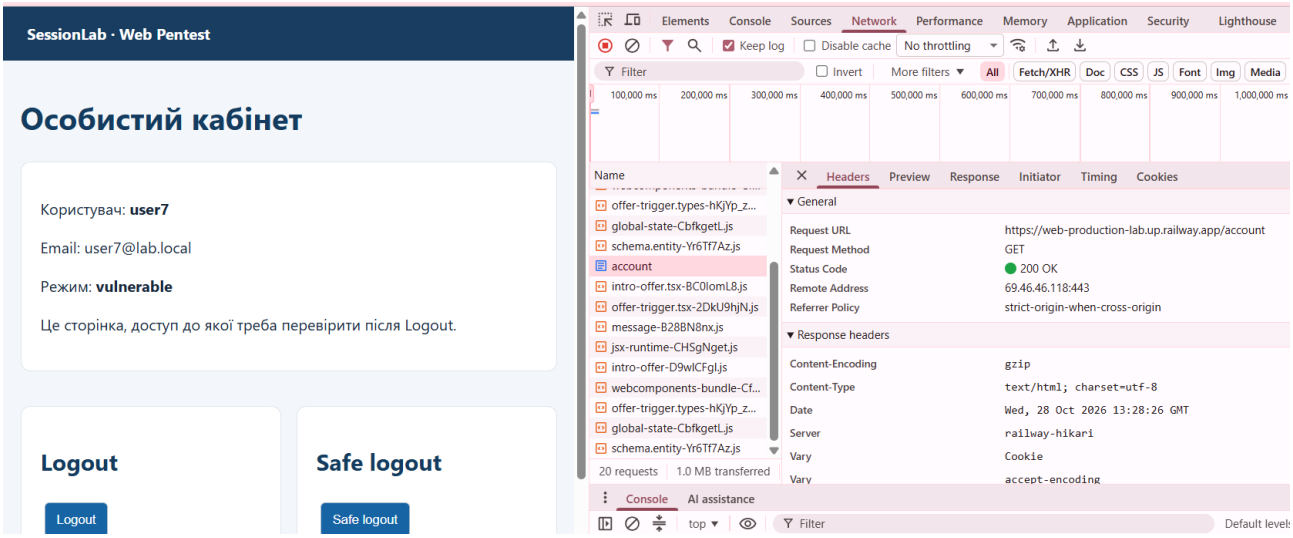
Ризик (Medium): самі по собі відсутні атрибути не дають доступ до акаунта, але вони полегшують викрадення sid разом з іншими проблемами, наприклад при XSS скрипт зможе прочитати cookie без HttpOnly, а без Secure sid можна перехопити в незахищеній мережі.

Рекомендація: для cookie сесії явно задавати HttpOnly, Secure і SameSite (Lax або Strict). Ці атрибути не замінюють захист від XSS і CSRF, тому їх треба використовувати разом із ним.

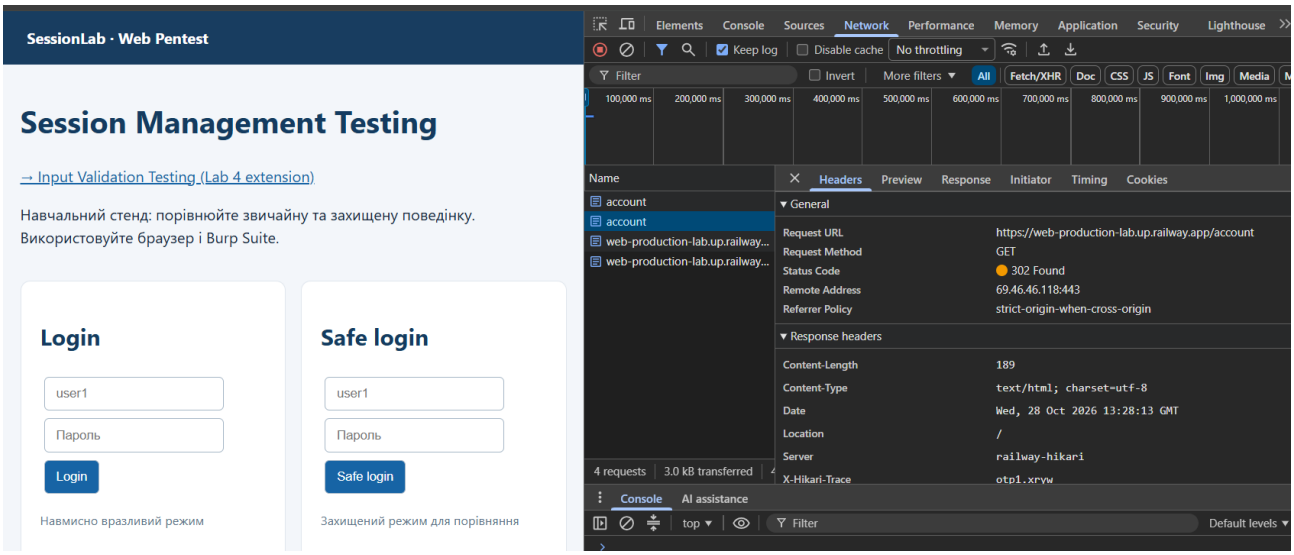
5 Session Timeout

Що перевіряла. Чи зберігається доступ до захищеної сторінки після очікування та чи відрізняється строк дії сесій у двох режимах.

Момент перевірки	**Vulnerable**	**Safe**
Близько 13:08	200 OK; кабінет доступний	200 OK; кабінет доступний
Близько 13:28	200 OK; user7	302 Found; Location: /



Vulnerable після очікування: /account повернув 200 OK і кабінет user7



Safe після очікування: /account повернув 302 Found та Location: /

Результат: о 13:08 в обох режимах /account давав 200 OK. О 13:28 у Vulnerable було так само 200 OK і кабінет user7, а у Safe вже 302 на головну сторінку.

Причина: у Safe на сервері задано обмеження часу життя або бездіяльності сесії, і за приблизно 20 хвилин воно спрацювало, а у Vulnerable за цей час сесія лишилась активною.

Ризик (Medium): надто довга сесія збільшує час, протягом якого викрадений sid або незакрите вікно браузера можуть бути використані.

Обмеження: я не визначала точний момент завершення сесії у Safe і не розрізняла тайм-аут бездіяльності та загальний час життя сесії, тому не називаю 20 хвилин точним налаштуванням. Те що у Vulnerable немає тайм-ауту взагалі цим тестом не доведено, показано лише різницю в перевіреному інтервалі.

Рекомендація: задати і перевіряти на сервері тайм-аут бездіяльності та максимальний час життя сесії залежно від вимог застосунку. Для точного результату повторити тест із запитами через рівні проміжки часу, щоб знайти момент, коли сесія перестає працювати.

6 Concurrent Sessions

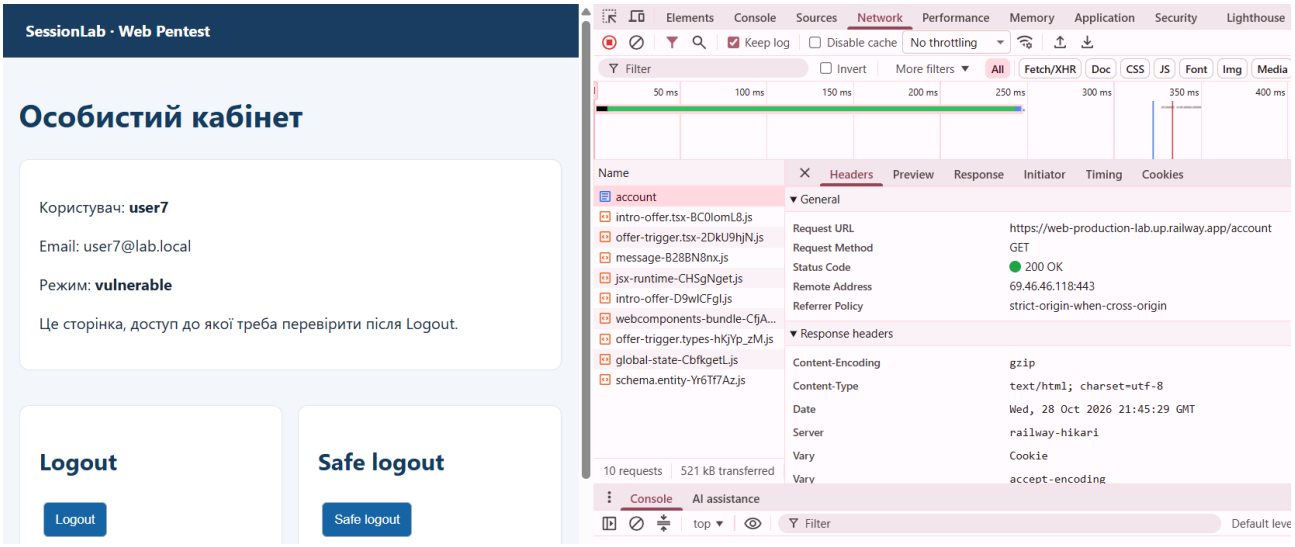
Що перевіряла. Чи залишається перша сесія активною після другого входу під тим самим обліковим записом, а також чи йдеться про різні сесії.

1.У вікні А звичайного Chrome увійшла як user7, відкрила /account і зберегла lab_sid.

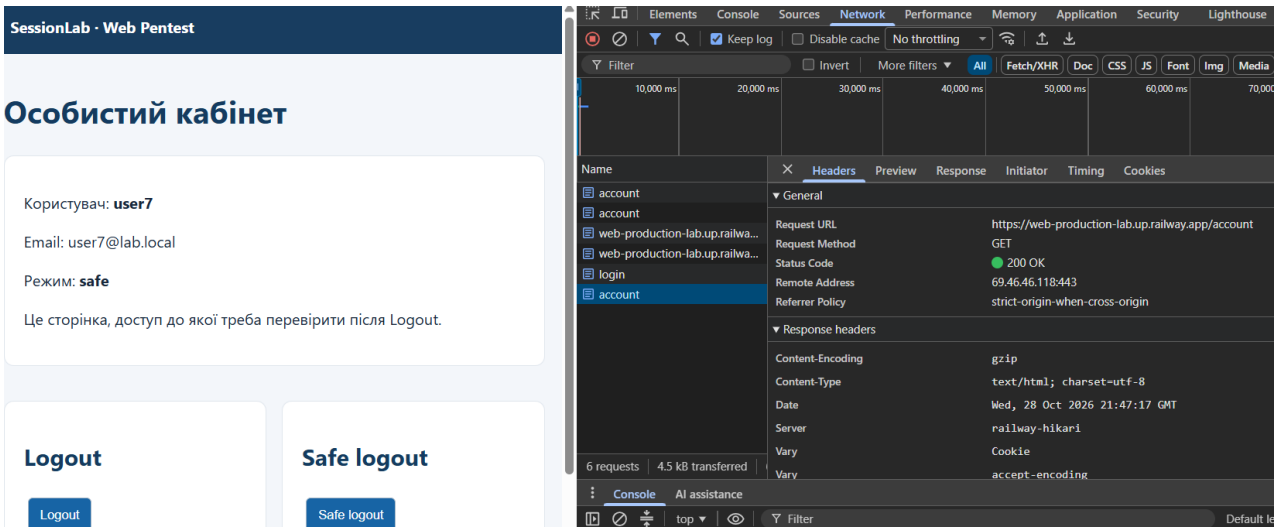
2.У вікні В в інкогніто увійшла тим самим користувачем та порівняла lab_sid. Значення у А та В були різними.

3.Після входу в В повернулася до А й оновила /account, а потім так само оновила сторінку у В.

4. Повторила перевірку через Safe login в обох вікнах. У Safe пара значень lab_sid також була різною.



Concurrent Sessions у Vulnerable: перша сесія А після другого входу продовжила відкривати /account з 200 OK.



Concurrent Sessions у Safe: перша сесія А після другого входу також продовжила відкривати кабінет user7.

Результат: і у Vulnerable, і в Safe другий вхід під user7 створив нову сесію з іншим sid, а перша сесія після цього продовжила відкривати /account (200 OK).

Причина: застосунок в обох режимах не обмежує кількість одночасних сесій одного користувача, тому різниці між Vulnerable і Safe тут немає.

Ризик (Low): паралельні входи самі по собі не є вразливістю, так як багато сервісів дозволяють заходити з кількох пристроїв. Ризик з'являється, якщо користувач не бачить свої активні сесії і не може їх закрити, тоді чужий вхід може довго лишатись непоміченим.

Обмеження: я не перевіряла максимальну кількість сесій, а також входи з різних IP-адрес і пристроїв.

Рекомендація: визначити правило для одночасних входів - якщо їх дозволено, показувати користувачу список активних сесій з можливістю завершити їх, а якщо встановлено ліміт, перевірити, що він дотримується.

Input Validation Testing

У цьому блоці спочатку перевіряла звичайні дані, потім передавала тестові рядки та порівнювала відповіді Vulnerable і Safe. У DevTools - Network - Payload перевіряла фактично переданий параметр і режим, щоб висновок не залежав лише від вигляду сторінки.

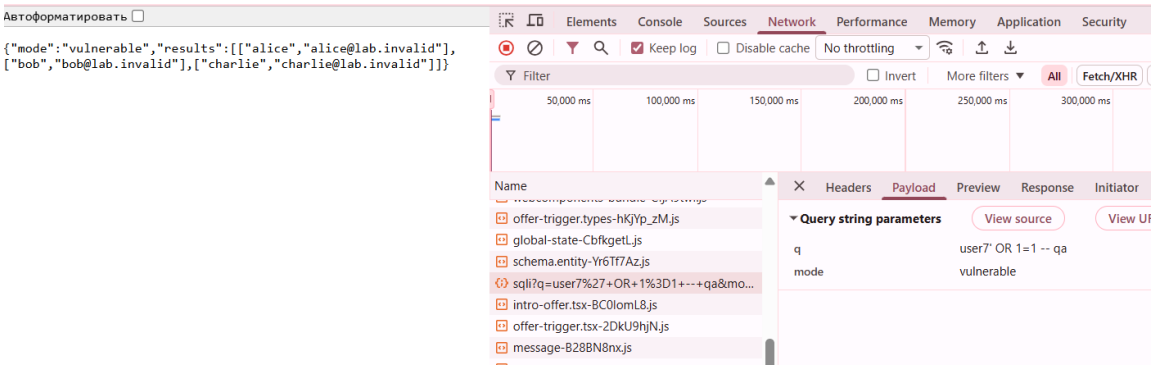
7 SQL Injection

Що перевіряла. Чи може значення параметра q у /lab/input/sqli змінити умову пошуку й повернути записи, яких немає у звичайній відповіді.

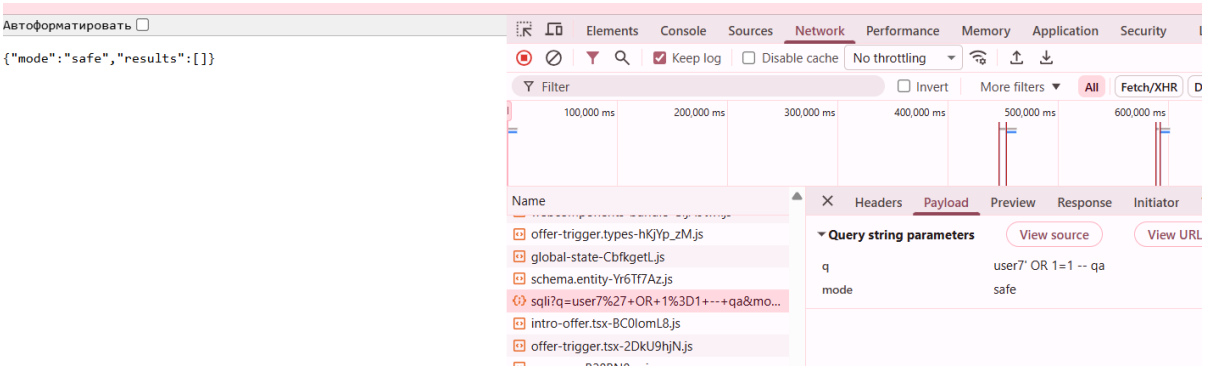
1. У Vulnerable виконала звичайний пошук user7 і отримала порожній список.

2. Передала q=user7' OR 1=1 -- qa, а потім змінила умову на q=user7' OR 1=2 -- qa, щоб порівняти істинний і хибний вирази. Повторила у Safe.

Дані перевірки	Vulnerable	Safe
user7' OR 1=1 -- qa	3 записи: alice, bob, charlie	results: []
user7' OR 1=2 -- qa	results: []	results: []
Звичайний пошук	user7 - results: []	alice - results: []



SQLi у Vulnerable: q=user7' OR 1=1 -- qa повернув 3 записи з іменами та email.



SQLi у Safe: той самий рядок з істинною умовою повернув порожній список.

Контрольний запит із хибною умовою у Vulnerable також повернув порожній список

q: user7' OR 1=2 -- qa; mode: vulnerable
{"mode":"vulnerable","results":[]}

Результат: у Vulnerable запит user7' OR 1=1 -- qa повернув три записи (alice, bob, charlie) з email, а запит з хибною умовою OR 1=2 повернув порожній список. У Safe обидва запити повернули порожній список.

Причина: у Vulnerable значення q вставляється прямо в текст SQL-запиту, тому ‘ закриває рядок, умова OR 1=1 стає завжди істинною, а -- відкидає решту запиту. Те що результат залежить від істинності умови, показує, що введення справді змінює логіку запиту. У Safe значення q передається як параметр і не може змінити структуру запиту.

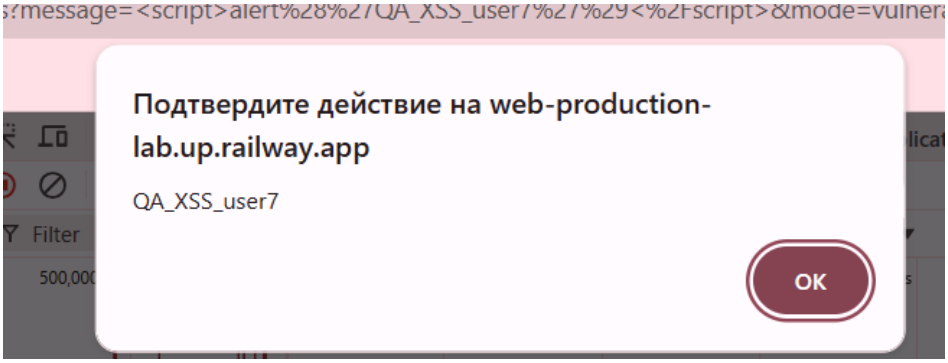
Ризик (High): у реальному застосунку SQL-ін'єкція може дозволити читати дані, до яких користувач не повинен мати доступу, наприклад усіх користувачів, а в гіршому випадку змінювати або видаляти дані.

Рекомендація: використовувати параметризовані запити, щоб q передавався як значення, а не як частина SQL, і давати обліковому запису бази даних лише необхідні права. Для Safe додатково виконати контрольний пошук з відомим існуючим записом.

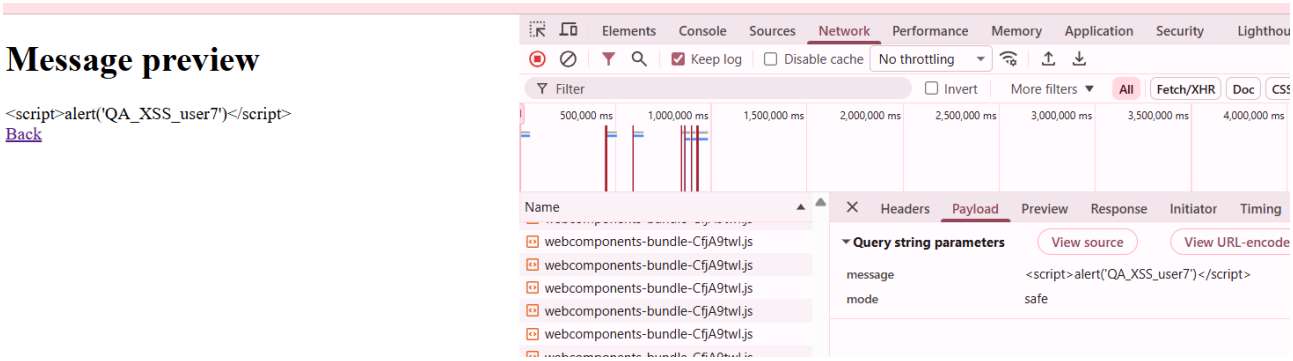
8 Cross Site Scripting

Що перевіряла. Чи виконається JavaScript, переданий у параметрі message форми /lab/input/xss, замість відображення як звичайного тексту.

1. У Vulnerable передала message=<script>alert('QA_XSS_user7')</script> та зафіксувала появу alert.
2. У Safe повторила той самий рядок зі script і переглянула як сторінку, так і HTML-відповідь у Network - Response.



XSS у Vulnerable: браузер виконав тестовий script та показав alert



XSS у Safe: видно той самий параметр message та mode=safe, а на сторінці тег показано як текст.

У HTML-відповіді Safe символи тегів були закодовані, тобто є безпечне відображення саме цього введення: <script>alert('QA_XSS_user7')</script>

Результат: у Vulnerable значення message зі скриптом повернулось у відповіді й виконалось у браузері (з'явився alert). У Safe той самий рядок відобразився як звичайний текст

Причина: у Vulnerable введення виводиться на сторінку без кодування, тому браузер сприймає його як код, а в Safe спеціальні символи перетворюються на безпечний текст. Це Reflected XSS, бо код передається в параметрі запиту і одразу повертається у відповіді.

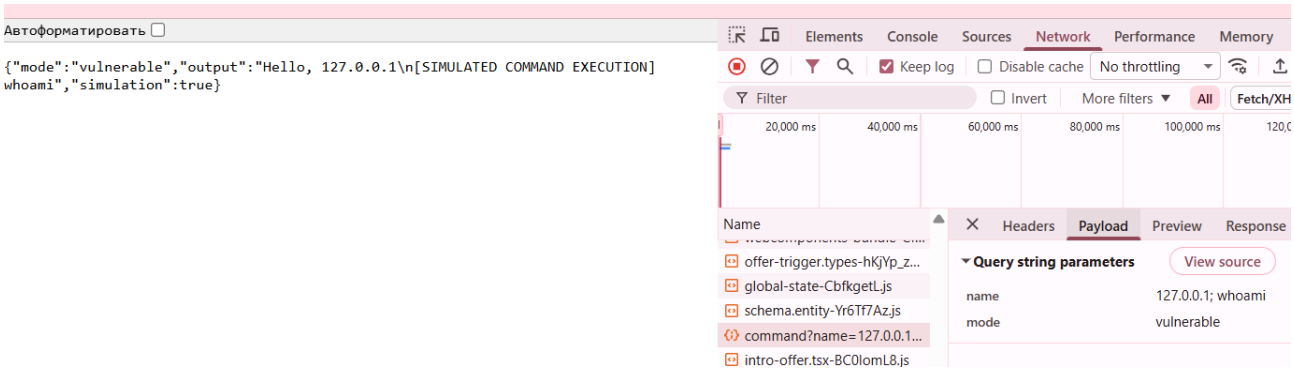
Ризик (High): у реальному застосунку скрипт може змінювати сторінку, виконувати дії від імені користувача і крадіжка даних, а якщо cookie без HttpOnly (як у Vulnerable), то й читати sid.

Рекомендація: кодувати дані при виводі відповідно до місця вставки (HTML, атрибут, JavaScript) і виводити їх як текст. Якщо HTML потрібен за функціоналом, використовувати перевірену бібліотеку очищення. Додатково ставити HttpOnly для cookie і налаштувати Content-Security-Policy, але це не замінює виправлення самого виводу.

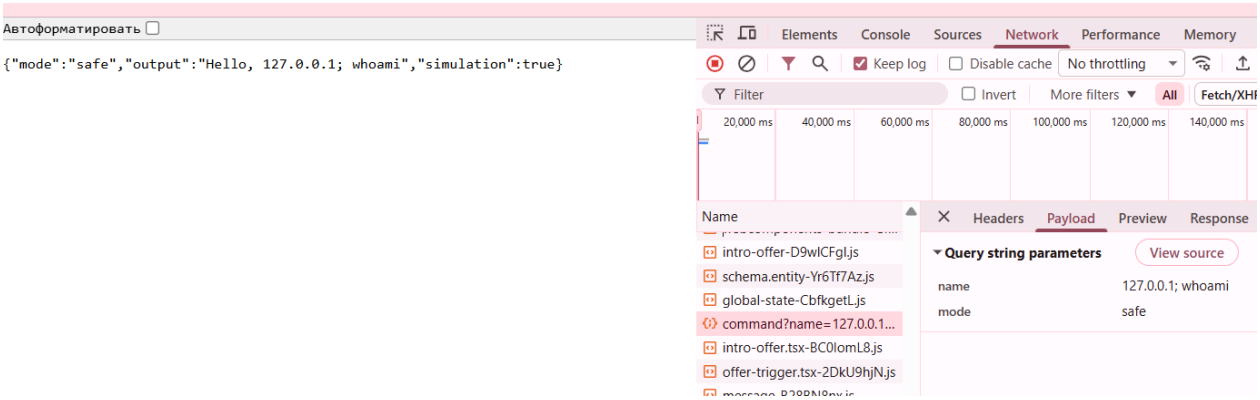
9 Command Injection

Що перевіряла. Чи змінює додавання команди у параметр name поведінку симулятора /lab/input/command та чи відрізняється обробка у Safe.

У Vulnerable написала 127.0.0.1 ; whoami та перевірила відповідь на name=127.0.0.1; whoami і повторила той самий рядок у Safe



Command Injection у Vulnerable: доданий рядок спричинив позначку [SIMULATED COMMAND EXECUTION], simulation має значення true.



Command Injection у Safe: увесь рядок 127.0.0.1, whoami залишився текстом у output.

Результат: у Vulnerable 127.0.0.1; whoami дало у відповіді [SIMULATED COMMAND EXECUTION] і whoami, а в Safe весь рядок залишився звичайним текстом

Причина: у Vulnerable символ ; працює як роздільник, і те що після нього, обробляється як окрема команда, а у Safe введення сприймається лише як дані, без розбору на команди.

Ризик (High): у реальному застосунку, якщо введення склеюється з командою оболонки, зловмисник може виконувати довільні команди з правами серверного процесу, і це може привести до повного контролю над сервером.

Рекомендація: не викликати команди оболонки з користувацьким введенням, а використовувати бібліотечні функції. Якщо запуск команди потрібен, передавати аргументи списком без оболонки і дозволяти лише перевірений формат введення, наприклад IP-адресу за шаблоном.

10 Path Traversal

Що перевіряла. Чи можна через параметр file в /lab/input/path вийти за межі дозволеного каталогу. Ручні спроби (./welcome.txt, ../welcome.txt, secret.txt, ../../etc/passwd) повернули File not found, тому запустила Burp Intruder на параметр file з навчальним списком із 50 payload у режимі vulnerable, а знайдений запит повторила в Repeater у Vulnerable і Safe.

Request	Payload	Status code	Response received	Error	Timeout	Length
25	php://filter/convert.base64-encode/resource=index.php	404	212			259
26	php://filter/read=string.rot13/resource=index.php	404	210			259
27	php://input	404	211			259
28	php://filter/convert.base64-encode/resource=config.php	404	213			259
29	php://filter/convert.base64-encode/resource=../config.php	404	212			259
30	C:\windows\win.ini	404	216			259
31	C:\windows\win.ini%00	404	217			259
32	..\..\..\..\..\..\..\..\windows\win.ini	404	211			259
33	../../../../../../../../windows/win.ini	404	230			259
34	C:\windows\system32\drivers\etc\hosts	404	209			259
35	..\..\..\..\..\..\..\..\windows\system32\drivers\etc\hosts	404	209			259
36	C:\windows\panther\unattend.xml	404	208			259
37	C:\Windows\debug\NetSetup.log	404	213			259
38	/var/log/apache2/access.log	404	233			259
39	/var/log/nginx/access.log	404	205			259
40	/var/log/httpd/access_log	404	209			259
41	/var/log/auth.log	404	207			259
42	../../../../../../../../var/log/apache2/access.log	404	206			259
43	../../../../../../../../var/log/nginx/access.log	404	208			259
44	~/.ssh/id_rsa	404	213			259
45	~/.ssh/authorized_keys	404	208			259
46	../../../../../../../../home/www/.ssh/id_rsa	404	209			259
47	.htaccess	404	228			259
48	config.php	404	210			259
49	config.txt	404	240			259
50	../config.txt	200	213			311
51	../config.php	404	208			259
52	index.php	404	206			259

../config.txt повернув 200 OK (довжина 311), решта 404.

Request

Pretty Raw Hex

```
1 GET /lab/input/path?file=../config.txt&mode=vulnerable HTTP/2
2 Host: web-production-lab.up.railway.app
3 Cookie: lab_sid=Fe86rxgfePRf6zqgJvgiPVamqvpb5PMDaSbh6uxhCjg
4 Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
5 Sec-Ch-Ua-Mobile: ?0
6 Sec-Ch-Ua-Platform: "Windows"
7 Accept-Language: ru-RU,ru;q=0.9
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
  (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
10 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,im
  age/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Sec-Fetch-Site: none
12 Sec-Fetch-Mode: navigate
13 Sec-Fetch-User: ?1
14 Sec-Fetch-Dest: document
15 Accept-Encoding: gzip, deflate, br
16 Priority: u=0, i
```

Response

Pretty Raw Hex Render

```
1 HTTP/2 200 OK
2 Content-Type: application/json
3 Date: Thu, 29 Oct 2026 12:06:34 GMT
4 Server: railway-hikari
5 X-Railway-Request-Id: 9_-36rS8SPe2NdRzjq40vQ
6 Content-Length: 86
7 X-Hikari-Trace: otp1.4qyp
8 X-Railway-Edge: otp1
9
10 {
    "content":"DEMO_API_KEY=not-a-real-secret",
    "file":"../config.txt",
    "simulation":true
  }
11
```

Vulnerable: file=../config.txt повернув 200 OK і вміст файлу, simulation: true

Request

Pretty Raw Hex

```
1 GET /lab/input/path?file=../config.txt&mode=safe HTTP/2
2 Host: web-production-lab.up.railway.app
3 Cookie: lab_sid=Fe86rxgfePRf6zqgJvgiPVamqvpb5PMDaSbh6uxhCjg
4 Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
5 Sec-Ch-Ua-Mobile: ?0
6 Sec-Ch-Ua-Platform: "Windows"
7 Accept-Language: ru-RU,ru;q=0.9
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
  (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
10 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,im
  age/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Sec-Fetch-Site: none
12 Sec-Fetch-Mode: navigate
13 Sec-Fetch-User: ?1
14 Sec-Fetch-Dest: document
15 Accept-Encoding: gzip, deflate, br
16 Priority: u=0, i
```

Response

Pretty Raw Hex Render

```
1 HTTP/2 404 Not Found
2 Content-Type: application/json
3 Date: Thu, 29 Oct 2026 12:06:48 GMT
4 Server: railway-hikari
5 X-Railway-Request-Id: oU-rfPNuRm0g_Avc6WHrDg
6 Content-Length: 27
7 X-Hikari-Trace: otp1.4qyp
8 X-Railway-Edge: otp1
9
10 {
    "error":"File not found"
  }
11
```

Safe: той самий запит повернув 404 і File not found

Результат: у Vulnerable запит file=../config.txt повернув 200 OK і вміст файлу config.txt, який лежить поза дозволеним каталогом (config.txt без ../ повернув File not found). У Safe той самий запит повернув 404 і File not found.

Причина: у Vulnerable значення file додається до шляху без перевірки, тому ../ піднімає шлях на рівень вище і застосунок віддає файл поза дозволеною папкою. У Safe такий шлях відхиляється.

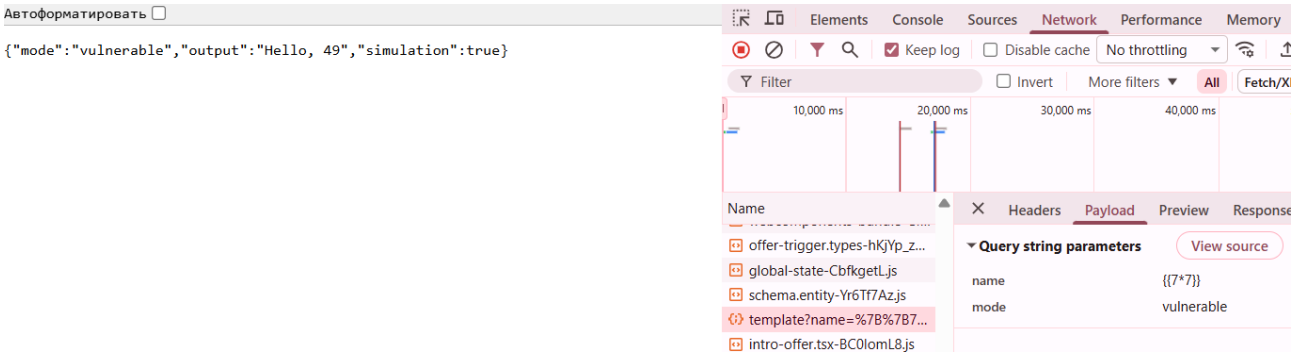
Ризик (High): у реальному застосунку це може дозволити читати файли поза дозволеною папкою, наприклад конфігурації, ключі або логи.

Рекомендація: дозволяти доступ лише до заздалегідь визначеного списку файлів, а шлях нормалізувати і перевіряти, що він лишається всередині дозволеної папки (CWE-22).

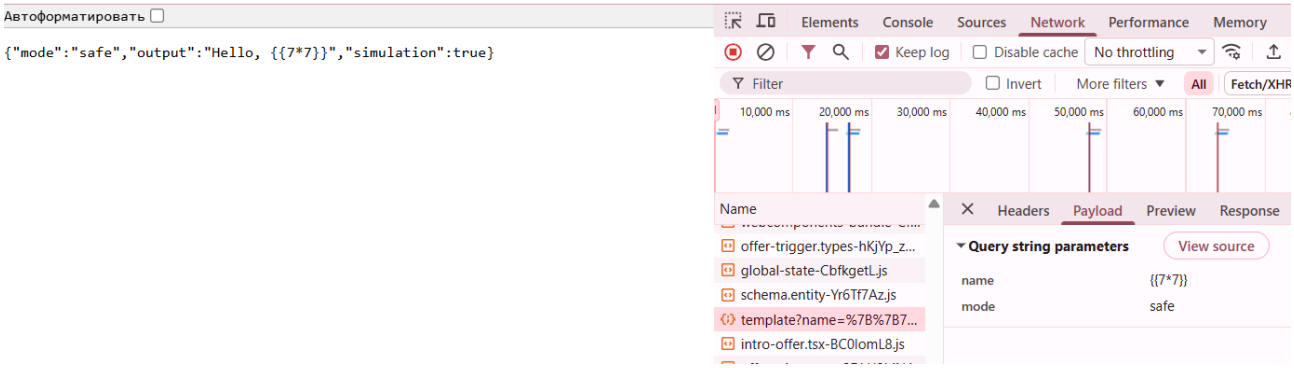
11 Server Side Template Injection

Що перевіряла. Чи обробляє симулятор /lab/input/template параметр name як вираз шаблону та чи залишає Safe це введення текстом.

1. Передала звичайне name=QA_SSTI_user7 у Vulnerable та Safe, щоб перевірити звичайне відображення введення.
2. Передала name={{7*7}} у Vulnerable та зафіксувала результат, після чого змінила вираз на {{7*8}}, щоб перевірити зміну обчисленого значення.
3. Повторила обидва вирази у Safe й перевірила name, mode, output і simulation у відповіді.



SSTI у Vulnerable: name={{7*7}} дав output Hello, 49, у відповіді є simulation: true.



SSTI у Safe: той самий вираз залишився текстом Hello, {{7*7}}.

Результат: у Vulnerable {{77}} дало Hello, 49, а {{78}} дало Hello, 56. У Safe обидва рядки залишились текстом (Hello, {{77}} і Hello, {{78}}).

Причина: у Vulnerable введення користувача стає частиною самого шаблону, і воно обчислюється, тому результат змінюється разом із виразом (49 і 56). У Safe введення передається як значення змінної в готовий шаблон і не обчислюється.

Ризик (High): у реальному застосунку SSTI залежно від рушія шаблонів може дозволити читати дані на сервері і виконувати код.

Рекомендація: використовувати заздалегідь написаний шаблон і передавати введення в нього тільки як значення змінної, а не будувати текст шаблону з даних користувача.

Висновки:

У Vulnerable підтверджено проблеми з сесіями: sid не змінюється після входу, старий sid працює після Logout, інша сесія лишається активною після зміни пароля, а cookie не має HttpOnly і Secure. Також підтверджено SQL Injection і Reflected XSS. Command Injection, Path Traversal і SSTI підтверджено в межах симуляції, бо справжнього виконання команд чи читання файлів сервера немає. У Safe всі ці перевірки пройшли правильно: sid змінюється, сесії завершуються, cookie має захисні атрибути, а введення не спрацьовує як код чи шлях. Причина проблем у Vulnerable в тому, що сервер не перевіряє sid і введення користувача. Одночасні сесії самі по собі не вразливість, а тест очікування показав різницю між режимами, але не дав точного тайм-ауту.

Подальші дії. Виправити в першу чергу сесії (новий sid після входу, завершення сесії на сервері), SQL-запити, вивід HTML і перевірку шляху до файлів. Потім задати атрибути cookie, час життя сесій і ліміт входів, після чого повторити тести.